Ders 20 Çalışma Özeti

Ders 20 Çalışma Özeti

Genel Konular

- Asıl konu yaklaşık 00:06'da başlar.
- Program, proses (process) ve thread kavramları
  - Program: diskteki derlenmiş kod. Proses: belleğe yüklenmiş, çalışan program. Thread: prosesin içindeki çalışma akışı.
  - Her prosesin en az bir thread'i vardır.
- İşlemcinin çalışması ve zaman paylaşımı
  - Komut döngüsü: fetch, decode, execute, write back.
  - Tek çekirdek bir anda tek komut yürütür. İşletim sistemi işlemciyi prosesler arasında zaman paylaşımıyla (time sharing) dağıtır.
  - Bir prosesten ötekine geçişe bağlam değiştirme (context switching) denir.
- Multitasking, multiprocessing ve multithreading
  - Multiprocessing: birden fazla prosesin aynı anda çalışması. Gerçek eş zamanlılık için çok çekirdek veya çok işlemci gerekir.
  - Multithreading: bir prosesin kendi içinde birden fazla thread'e bölünmesi.
  - Eş zamanlı (concurrent) çalışma ve paralelleştirme örneği: iki dizinin eleman eleman toplanması.
- Proses kontrol bloğu (PCB) ve thread kontrol bloğu (TCB)
  - PCB'de program sayacı, proses durumu, PID, bellek bilgileri gibi bilgiler tutulur.
  - Her thread'in kendi TCB'si vardır. Thread'ler prosesin ortak alanlarını (ör. global değişkenler) paylaşır.
- Thread durumları
  - New, Runnable, Running, Blocked/Waiting (I/O, sleep, wait, join), Dead.
  - Busy waiting yerine `sleep` veya `wait` kullanılmalı.
- Java'da thread oluşturma
  - `Runnable` arayüzünü uygulayan sınıf, `run()` metodunu yazar.
  - `new Thread(runnable).start()` ile thread başlatılır.
  - `Thread` sınıfından kalıtım ve anonim iç sınıf (anonymous inner class) ile thread oluşturma da gösterildi.
  - Örnekler: maç izleyen taraftarlar (MatchV1-V4), iki işçi, `join`, thread adı, öncelik (priority).
- JUnit ile birim test (unit test)
  - JUnit 3 ve JUnit 4 farkı. Fowler'ın video kiralama (Movie, Rental, Customer) örneği üzerinden `assertEquals`.
- Thread havuzu (thread pool)
  - `Executor`, `ExecutorService`, `Executors.newFixedThreadPool`, `newCachedThreadPool`.
  - `shutdown()`, `shutdownNow()`, `isShutdown()`, `isTerminated()`.
- Yarış durumu (race condition) ve senkronizasyon
  - Klasik problemler: yemek yiyen filozoflar (dining philosophers), üretici-tüketici (producer-consumer), tek şeritli köprü.
  - Kilitlenme (deadlock) ve açlık (starvation).
  - Banka hesabı simülasyonu: 100 kez 1 yatırma, 50 kez 1 çekme. Senkronizasyonsuz sonuç 50 yerine 4, 6, 8 gibi değerler çıktı.
  - Çözüm: `synchronized` metotlar.
- Thread-safe veri yapıları: `Vector`, `Hashtable`.
- Ders sonu: ASP.NET MVC projesine kısa giriş (Visual Studio). Gelecek hafta MVC anlatı-

lacak.

Hocanın Özellikle Vurguladığı Kısımlar

- Thread'i `run()` ile değil, `start()` ile başlatın.
  - `run()` doğrudan çağrılırsa normal metot çağrısı olur, kod sıralı çalışır.
- `Thread` sınıfından kalıtım yerine `Runnable` arayüzünü uygulayın.
  - Gereksiz kalıtım fazladan yük getirir.
- Thread'lerin çalışma sırası belli değildir.
  - Aynı programı her çalıştırmada farklı çıktı görebilirsiniz.
- Global (paylaşılan) değişken kullanmayın.
  - Birden fazla thread aynı değişkeni değiştirirse yanlış sonuç çıkar.
- Boş döngüyle bekleme (busy waiting) yapmayın.
  - Thread'i `sleep` veya `wait` ile bloklayın. Boş döngü işlemciyi boşuna harcar.
- Program, proses, thread, context switching, multitasking, multiprocessing ve multithreading kavramlarını bilin.
  - Bunlar gelecek dönem İşletim Sistemleri dersinde ayrıntılı görülecek.

Kısa Tekrar Notları

- Proses: çalışan programın bellekteki kopyası. İşletim sistemi onu PID ile yönetir.
- Thread: prosesin içindeki yürütme akışı. Proses en az bir thread içerir (main thread).
- Context switching: çalışan prosesin/thread'in durumunu kaydedip başkasına geçmek.
- Multiprocessing için gerçek paralellik çok çekirdek ister. Tek çekirdekte yalnızca hızlı geçişle sanal paralellik olur.
- Java'da thread: `class X implements Runnable { public void run() {...} }`, sonra `new Thread(new X()).start()`.
- `t.join()`: çağıran thread, `t` bitene kadar bekler.
- `Thread.sleep(ms)` checked exception olan `InterruptedException` fırlatabilir. `try-catch` veya `throws` gerekir.
- Öncelik: `Thread.MIN_PRIORITY` = 1, `NORM_PRIORITY` = 5, `MAX_PRIORITY` = 10. Öncelik yalnızca bir ipucudur; sıra garanti değildir.
- Thread havuzu: `ExecutorService ex = Executors.newCachedThreadPool(); ex.execute(task); ex.shutdown();`
- `shutdown()`: yeni görev almaz, çalışanların bitmesine izin verir. `shutdownNow()`: çalışanları kesmeye çalışır, başlamamış görevlerin listesini döndürür.
- Race condition: paylaşılan veriye eş zamanlı erişimde sonucun erişim sırasına bağlı olması.
- `synchronized` metot: aynı nesne üzerinde aynı anda yalnızca bir thread bu metodu çalıştırabilir.
- Deadlock: thread'lerin birbirini sonsuza kadar beklemesi. Starvation: bir thread'e hiç sıra gelmemesi.
- JUnit 4: test metodu `@Test` ile işaretlenir, `assertEquals(beklenen, gerçek)` ile karşılaştırılır.

Detaylı Açıklamalar

Runnable ile thread oluşturma

Hocanın taraftar örneğindeki yapı şöyledir. Her taraftar 10 kez bağırır, her bağırıştan sonra 0-1 saniye arası rastgele uyur.

```
public class SportsFan implements Runnable {
    private final String name;
    private final String slogan;

    public SportsFan(String name, String slogan) {
        this.name = name;
        this.slogan = slogan;
    }
```

```
    @Override
    public void run() {
        try {
            for (int i = 0; i < 10; i++) {
                System.out.println(name + ": " + slogan);
                Thread.sleep((long) (Math.random() * 1000));
            }
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

// Test sınıfında:
Thread t1 = new Thread(new SportsFan("Ali", "GS is the champion"));
Thread t2 = new Thread(new SportsFan("Veli", "FB is the best"));
t1.start();
t2.start();
```

- `start()` yeni bir thread açar ve `run()` metodunu o thread'de çalıştırır.
- Çıktıdaki sıra her çalıştırmada değişir. Derste "FB is the best" bazen en sonda, bazen başta çıktı.

join ve öncelik

- `t1.start(); t1.join();` yazılırsa main thread, `t1` bitmeden sonraki satırlara geçmez.
- `join(ms)` belirli bir süre bekler, süre dolunca main devam eder.
- Derste minimum ve maksimum öncelikli iki thread çalıştırıldı. Yüksek öncelikli olan çoğunlukla önce bitti. Bazen düşük öncelikli de önce girebilir.

Thread havuzu ile banka simülasyonu

Ekrandaki `BankSimulation` kodunun yapısı:

```
Account anAccount = new Account(0);
System.out.println("Before: " + anAccount.getBalance());
ExecutorService executor = Executors.newCachedThreadPool();
for (int i = 0; i < 100; i++)
    executor.execute(new AccountDepositor(anAccount, 1));
for (int i = 0; i < 50; i++)
    executor.execute(new AccountWithdrawer(anAccount, 1));
executor.shutdown();
while (!executor.isTerminated()) { }
System.out.println("After: " + anAccount.getBalance());
```

- Beklenen sonuç 50. `deposit` ve `withdraw` senkronize değilken derste 6, 4, 8 gibi değerler çıktı.
- `Account` sınıfında `deposit`, `withdraw` ve `getBalance` metotları `synchronized` yapılınca sonuç 50 oldu.

Doğrulanmış senkronize sürüm (Java 25 ile çalıştırıldı, çıktı `Before: 0.0`, `After: 50.0`). Bu kısa sürümde gecikme yoktur; derste kullanılan sürümde ara değişkenden sonra gecikme vardı:

```
class Account {
    private double balance;

    Account(double b) { balance = b; }

    synchronized double getBalance() { return balance; }

    synchronized void deposit(double amt) {
```

```
        double newBalance = balance + amt;
        balance = newBalance;
    }

    synchronized void withdraw(double amt) {
        double newBalance = balance - amt;
        balance = newBalance;
    }
}
```

Neden yanlış sonuç çıkıyor?

- `balance = balance + amount` tek bir işlem değildir. İşlemci önce değeri okur, sonra toplar, sonra geri yazar.
- Alice ve Bob örneği: hesapta 1000 TL var. Alice 100, Bob 50 çekiyor. Beklenen sonuç 850.
  - İkisi de 1000'i kendi thread'ine kopyalar.
  - Alice 900 yazar. Bob, elindeki 1000'den 50 düşüp 950 yazar.
  - Sonuç 950 olur. Alice'in çekimi kaybolmuştur (lost update).
- Basit bir `lock` bayrağı da tam çözüm değildir. Bayrağı okuma ve değiştirme ayrı adımlar olduğu için iki thread aynı anda "boş" görüp girebilir.
- Çözümler: semafor (semaphore), monitör. Java'nın `synchronized` anahtar kelimesi monitör mekanizmasıdır.

JUnit ile birim test

- Amaç: sınıfın metotlarını `main` içinde gözle kontrol etmek yerine otomatik test etmek.
- JUnit 3: test sınıfı `TestCase`'ten kalıtım alır. Test metotlarının adı `test` ile başlar. `setUp()` her testten önce çalışır. `TestSuite` ile testler birleştirilir.
- JUnit 4: kalıtım gerekmez. Test metodu `@Test` ile işaretlenir.
- Fowler'ın örneği: `Customer` sınıfının `getName()` ve `statement()` metotları test edilir. Aşağıdaki kod JUnit 4 biçiminde temsilî bir örnektir; derste slayttaki JUnit 3 sürümü gösterildi.

```
@Test
public void testGetName() {
    Customer c = new Customer("Yunus");
    assertEquals("Yunus", c.getName());
}
```

- NetBeans'te sınıfa sağ tıklayıp test dosyası oluşturulabilir. Sonuçta testlerin yüzde kaçının geçtiği görülür.

Ders sonrası notlar

- **"Frekans paylaşımı":** Hoca işlemci paylaşımını "zaman paylaşımı ve frekans paylaşımı" diye ikiye ayırdı. İşlemci zamanlamasında frekans paylaşımı diye bir yöntem yoktur. Frekans paylaşımı (FDM) haberleşme kanallarında kullanılır. İşlemci için geçerli olan zaman paylaşımıdır.
- **Preemptive ve non-preemptive:** Derste preemptive için "kaynağı tutunca bırakana kadar başkasına izin vermiyor" dendi. Bu, non-preemptive (kesintisiz) zamanlamanın tanımıdır. Doğrusu:
  - Preemptive (kesintili): işletim sistemi çalışan prosesi süresi dolunca veya daha öncelikli bir proses gelince işlemciden çıkarabilir.
  - Non-preemptive (kesintisiz): proses işlemciyi kendisi bırakana (bitene veya bloklanana) kadar kullanır.
- **`synchronized` neyi kilitler?** Derste "metodun kullandığı bütün kaynaklar bloke ediliyor" dendi. Doğrusu: `synchronized` metot, o nesnenin monitörünü (kilidini) alır. Yalnızca aynı nesne üzerindeki diğer `synchronized` metot ve bloklar beklemeye geçer. Senkronize olmayan metotlar aynı alana yine erişebilir.

- **İstisna adı:** Altyazıda “uninterrupted exception” diye geçiyor. Doğru ad `InterruptedException`. Checked bir istisnadır. Hocanın dediği gibi “uyanmama” için değildir; uyuyan thread başka bir thread tarafından kesilince oluşur.
- **“Java’da da aynıydı, void pointer”:** Hoca burada C’deki pthread fonksiyonunu kastediyor. C’de thread fonksiyonu `void *` alır ve döndürür. Java’da `run()` parametre almaz ve `void` döndürür.
- **`run()` içinden checked exception:** `run()` imzası `throws` içeremez. Sonuç veya checked exception döndürmek gerekiyorsa `Callable<V>` ve `ExecutorService.submit()` kullanılabilir. Derste bu konuya girilmedi.
- **`shutdownNow()`:** Derste “tamamlanmamış thread’leri kapatıyor ve listesini döndürüyor” dendi. Doğrusu: çalışan görevleri `interrupt` ile durdurmaya çalışır ve hiç başlamamış görevlerin (`List<Runnable>`) listesini döndürür. Görev kesilmeye tepki vermezse çalışmaya devam edebilir.
- **Üretici-tüketici:** Altyazıda “kaynak doluyken prodüsür tüketemez” geçiyor. Üretici tüketmez; doğrusu kaynak doluyken üretemez.
- **Son yazan belirler:** Hoca Alice-Bob örneğinde “şu önce yazarsa 900, şu önce yazarsa 850” dedi. İki thread de eski değeri okuduysa son yazan belirler; sonuç 900 ya da 950 olur. 850 ancak işlemler karışmazsa çıkar.
- **`static` gerekmez:** Hoca değişkenin görülmesi için `static` olmasını gerekçe gösterdi. Alice ve Bob aynı nesneyi paylaştığı için `static` olmayan alan da ikisince görülür (Java ile denendi).
- **Bekleme döngüsü:** Banka simülasyonundaki `while (!executor.isTerminated()) { }` satırı busy waiting’dir. Hocanın “busy waiting yapmayın” uyarısına uygun olarak bunun yerine `executor.awaitTermination(1, TimeUnit.MINUTES)` kullanmak daha doğrudur.
- **Thread-safe koleksiyonlar:** `Vector` ve `Hashtable` eski sınıflardır. Bugün genellikle `ConcurrentHashMap`, `CopyOnWriteArrayList` veya `Collections.synchronizedList(...)` tercih edilir.
- **Alice-Bob demosu:** Derste kodun ekrandaki sürümünde sonuç 1950 ve 1900 olarak çıktı. Mantık aynıdır: senkronizasyon yoksa çekimlerden biri kaybolabilir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.